C

CERTICHAIN

CERTIFICATE INTEGRITY LAB

BLOCKCHAIN-INSPIRED CREDENTIAL VERIFICATION

CertiChain

A working demonstration of digital certificate fingerprints, hash-linked records, and tamper detection.

Abstract. CertiChain is an educational software prototype that fingerprints certificate files with SHA-256, compares them against a registry of fictional sample credentials, and records issuance and verification events in a hash-linked ledger. It includes one-click demo credential generation, exact-file verification, an explorer for inspecting record hashes, and a safe tamper-detection simulation. The project demonstrates how cryptographic integrity checks work while making clear that a local hash-linked ledger is not a production or decentralized blockchain.

PROJECT TYPE	Interactive web application / educational prototype
CORE TECHNOLOGY	Python · Streamlit · SHA-256 · JSON ledger · PDF generation
DEMO CONTENT	3 preloaded fictional certificates + automatic demo issuance
KEY LEARNING	File integrity, hash linking, audit events, and technical limitations

Core idea: a matching fingerprint identifies exact file bytes in the current registry; linked event hashes make record changes detectable when the chain is recomputed.

Prepared as a classroom project and demonstration. All bundled and generated credentials are fictional samples.

1. Problem and objectives

Digital certificates can be copied, modified, or presented without reliable provenance. A visual inspection can show what a certificate says, but it does not establish that the file matches a previously registered version. This prototype uses a repeatable fingerprint and an inspectable record sequence to demonstrate one part of the verification problem.

Project objectives

- Calculate SHA-256 fingerprints from exact certificate file bytes.
- Maintain a small registry of fictional PDF credentials and support one-click demo issuance.
- Compare uploaded file hashes with registered fingerprints and show a clear match/no-match outcome.
- Record issuance and verification activity as blocks connected by previous-hash references.
- Recompute both block hashes and registry metadata digests to detect inconsistent changes.
- Explain where file integrity ends and real-world issuer authentication begins.

Scope boundaries

The system is designed to teach the concepts rather than replace a university or government credential service. A successful match establishes that the bytes match a file registered in this demo system. It does not establish a real issuer's identity, the recipient's identity, or the legal status of a qualification.

Technology stack

COMPONENT	ROLE
Python + Streamlit	Application logic, navigation, form-free demo workflows, and interactive dashboard.
SHA-256 / hashlib	Creates file fingerprints and stable digests for canonicalized records.
Canonical JSON	Serializes metadata in a consistent order before it is hashed.
ReportLab + QR library	Generates polished fictional PDF certificates and metadata-only QR codes.
JSON ledger	Persists registry entries and the hash-linked event history on the running host.
unittest	Checks expected matching, chain failure on alteration, registry checks, and persistence.

Design decision: the user does not need to enter a large set of fields. Sample certificates are preloaded, and the issue-demo action generates a fictional credential automatically.

2. System architecture and workflow

CertiChain separates the core hashing and ledger operations from the user interface. The Streamlit app invokes reusable core functions to create fingerprints, register credentials, append events, and validate integrity.

1. INPUT	2. HASH	3. COMPARE	4. RECORD	5. AUDIT
Sample PDF, generated PDF, or uploaded file	SHA-256 over exact bytes	Compare to registry fingerprints	Append result with previous block hash	Recompute block hashes and registry references

Certificate issuance

The app creates fictional metadata, renders a PDF, hashes the final PDF bytes, and stores a registry entry that includes its digest. It then appends a registration event that includes the certificate ID, file hash, and a digest of selected registry metadata. The PDF is downloadable and is marked as a demonstration sample rather than an official qualification.

Certificate verification

When a user uploads a file, the app computes its SHA-256 digest and compares that digest with registered fingerprints. If a fingerprint matches, the app identifies the corresponding registry entry. If there is no match, the app reports that the file is not an exact match and records the result without saving the uploaded file contents.

Hash-linked ledger

Each block contains an index, UTC timestamp, event type, details, previous block hash, and current block hash. The genesis block points to an all-zero value. All later blocks reference the prior block's computed hash. The explorer recomputes each block hash and checks every previous-hash link. A separate registry check compares registry metadata digests to their registration events.

Exact-byte matching is deliberately strict: re-exporting or scanning a PDF can alter its bytes and produce a no-match even if the visible text seems identical.

3. Implementation and testing

The project includes automated tests for the hashing and event-ledger logic. The tests exercise the happy path as well as inconsistent records; they do not replace a full security assessment or integration testing on the final hosting platform.

TEST	EXPECTED OUTCOME	PURPOSE
Initial samples	3 credentials registered; chain and registry pass	Confirms bundled PDFs and initial event sequence are consistent.
Exact sample file	Computed hash equals registered hash	Confirms uploaded/sample bytes are matched by fingerprint.
Alter block details	Chain integrity fails	Shows an edited record no longer agrees with its stored digest.
Alter registry metadata	Registry check fails	Shows metadata changes are checked against its digest/event.
Unknown file	No match; event is still recorded	Covers the unsuccessful verification path.
New generated PDF	Registered fingerprint matches generated bytes	Checks one-click issuance and registration logic.
Tamper simulation	Altered copy fails; saved ledger unchanged	Demonstrates chain tamper detection without corrupting the real demo data.

Observed automated test result

Core test suite: 8 tests passed in the build environment. Tests cover initial sample registration, PDF fingerprints, chain alteration, registry alteration, verification recording, unknown-file handling, one-click PDF registration, and atomic JSON writes.

Data handling

Verification uploads are processed by the running application to calculate a digest. The verification action records the filename, hash, result, and matched ID if present; it does not save the uploaded file contents. Generated demonstration PDFs and the local JSON ledger are kept in the app's data directory on the running host.

Operational consideration

Local JSON is suitable for a short classroom prototype, not a production registry. Streamlit Community Cloud's local filesystem is not durable ledger storage and should not be used for private certificates. Concurrent writes, backup and recovery, access controls, and deployment persistence would require a more robust data layer.

4. Limitations, future work, and conclusion

The most important design distinction is between **file integrity** and **credential authenticity**. A hash can identify exact bytes relative to a registry; it cannot establish which real-world organisation created a credential unless the system also has a trustworthy issuer model.

LIMITATION	WHY IT MATTERS	FUTURE DIRECTION
No decentralised consensus	One authority controls the local JSON file and can rewrite all records.	Use independently operated validators or a suitable established ledger if distributed trust is required.
No issuer authentication	A match does not prove an organisation or recipient is genuine.	Onboard trusted issuers, authenticate accounts, and verify digital signatures.
Exact-byte dependence	A re-exported or scanned copy may not match the original bytes.	Design a signed credential schema and robust verification workflow around the credential ID and issuer signature.
Limited lifecycle management	No formal revocation, expiry, or issuer status checks are implemented.	Add revocation status, validity periods, key rotation, and audit procedures.
Privacy and storage	A public demo should not receive sensitive personal credentials; cloud local storage can reset.	Apply data minimisation, consent, retention policies, access controls, and durable secured storage.

Suggested three-minute demonstration

- Dashboard: introduce the preloaded registry and current integrity status.
- Verify a bundled sample and show its SHA-256 digest and new ledger receipt.
- Issue a fictional certificate, download it, and upload it to show a match.
- Open the ledger explorer and run the simulated tampering test.
- Close by distinguishing integrity checking from issuer authentication and decentralised consensus.

Conclusion

CertiChain makes the core ideas of cryptographic fingerprinting and hash-linked events observable in an interactive app. It demonstrates how a registry can identify exact file content and how record changes can be detected by recomputing hashes. Its classroom value also comes from its boundaries: meaningful credential trust needs issuer identity, governance, privacy protection, revocation handling, and a clear accountability model—not just a blockchain label.

Key takeaway: the technology is useful only when the trust model is explicit. CertiChain teaches a real cryptographic mechanism without overstating what the prototype guarantees.

Supporting files included with the project: source code, requirements, sample PDF certificates, automated tests, setup instructions, and presentation notes.